



DevOps

THE LANDSCAPE · DAY 1

Every tool in this course is an answer to a problem.

It works on my machine.

The failure that gave birth to DevOps



"It ran fine when I built it. Ship it."

THE
WALL

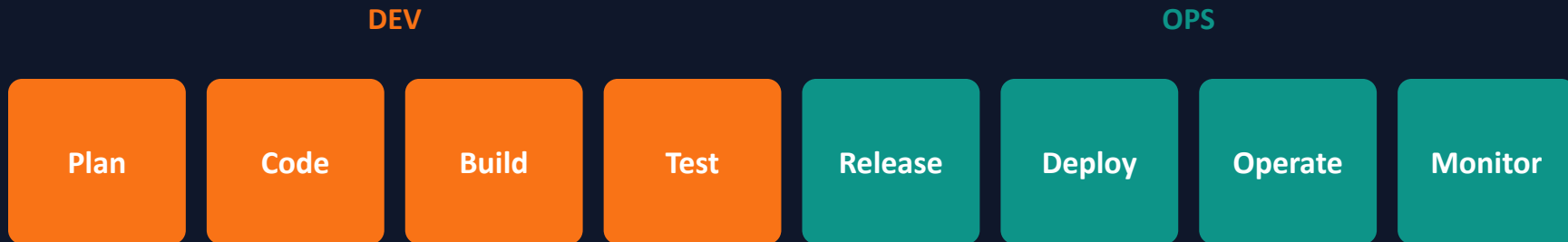


"It's broken in production. Not our code."

DevOps = tear down the wall. Shared ownership + automation, so software ships fast AND reliably.

The DevOps Loop

Everything you'll learn lives somewhere on this loop — it never stops.



↻ *the output of Monitor feeds straight back into Plan — continuously*

In every future class I'll point to where we are on this loop.

01 Version Control

"I lost my work — and we keep overwriting each other's code."



THE PROBLEM

Two people edit the same file and one save destroys the other's work. You delete something on Friday and there's no way to get it back. No history, no parallel work, no safety net.



THE IDEA THAT FIXES IT

Track every change as a recoverable snapshot, and let many people work in parallel on the same code without stepping on each other.

TOOLS YOU'LL MEET

Git

GitHub

GitLab

SVN (old)

Remember: Git is NOT GitHub

02 Containers

"It works on my machine... but it dies on the server."



THE PROBLEM

The app runs on the developer's laptop and breaks in production different OS, a missing library, the wrong version of something. The environment is never quite the same twice.



THE IDEA THAT FIXES IT

Package the app together with its entire environment into one portable unit that runs identically everywhere your laptop, a teammate's, the server.

TOOLS YOU'LL MEET

Docker

containerd

Podman

Lighter than a full VM — shares the host kernel

03 Container Orchestration

"50 containers across 10 machines — and one just died at 3 a.m."



THE PROBLEM

One container is easy. Running hundreds across a fleet — restarting dead ones, scaling under load, rolling out updates with zero downtime, wiring them together — is impossible to do by hand.



THE IDEA THAT FIXES IT

Declare your desired state ("I want 5 healthy copies of this") and a control system continuously makes reality match it — healing, scaling, and updating on its own.

TOOLS YOU'LL MEET

Kubernetes

Docker Swarm

EKS / GKE

The hardest topic — and it's late on purpose

04 CI / CD

"Deploying is a scary manual ritual nobody wants to do on a Friday."



THE PROBLEM

Every release is a human running steps by hand — build, test, copy, restart. Someone forgets a step, prod breaks, and it's 6 p.m. on a Friday. Slow, error-prone, and stressful.



THE IDEA THAT FIXES IT

Automate the whole path from "code pushed" to "tested and deployed" so releasing becomes boring and repeatable. CI = auto-build & test every change. CD = auto-ship it.

TOOLS YOU'LL MEET

Jenkins

GitHub Actions

GitLab CI

This is the spine the whole course builds toward

05 Config Management & Infra as Code

"I set up that server by hand months ago — and now I can't reproduce it."



THE PROBLEM

Servers configured manually become unique 'snowflakes' nobody can rebuild. When one dies, or you need ten more just like it, you're stuck reconstructing magic from memory.



THE IDEA THAT FIXES IT

Describe your infrastructure and configuration AS CODE — version it, review it, and run it to recreate everything from scratch, identically, on demand.

TOOLS YOU'LL MEET

Ansible

Terraform

OpenTofu

CloudFormation

Configure existing machines vs provision the infra itself

06 Monitoring & Observability

"We only found out the site was down... from angry users."



THE PROBLEM

You're flying blind. There's no view into whether things are healthy, fast, or failing — so your users become your alerting system, and you're always reacting after the damage is done.



THE IDEA THAT FIXES IT

Continuously collect metrics and logs, visualize them on dashboards, and get alerted automatically — so you catch problems BEFORE your users ever notice.

TOOLS YOU'LL MEET

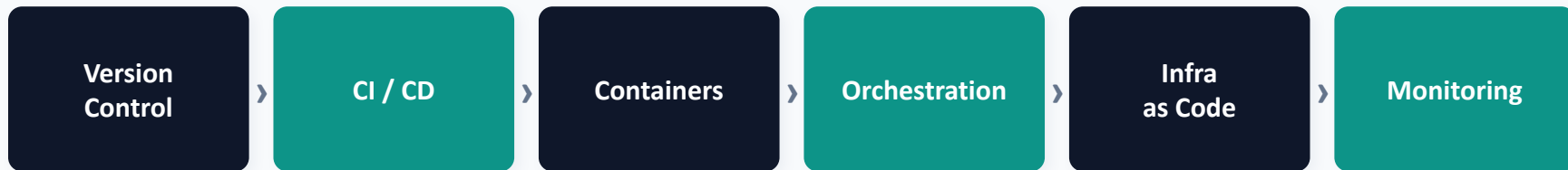
Prometheus

Grafana

Prometheus collects · Grafana visualizes

Six problems. One chain.

They don't just sit side by side — each one feeds the next.



Code is versioned, a pipeline builds & tests it, the result is a container, orchestration runs it, infra-as-code provisions the ground it runs on, and monitoring watches the whole thing.

The lesson that outlasts every tool: when a new tool appears in five years, don't panic — just ask “which of these problems does it solve?” and slot it in.

Before next class

Small, but do it — everything in Chapter 5 builds on this.

1

Create a GitHub account

Free. Use a name you don't mind being public — it's your portfolio later.

2

Make one repository and put a single file in it

Any way you can — the web upload button counts. The goal is just to have touched it once.

Bring questions. Next time we go deeper on the lifecycle and get your environment set up.